

SimpleFaceMe Architecture

Revision History

- 2026-05-13: Initial architecture definition and configuration decisions

Configuration Decisions

Hardware Interfaces:

- UART for LD2450: Default ESP32-S3 pins (demo - exact pins TBD)
- I2C for PCA9685: Default SDA/SCL pins

LD2450 Configuration:

- Keep it simple - basic single-target tracking mode
- No complex zone configuration initially
- Target selection: Closest target when multiple detected

Servo Behavior:

- Update rate: Configurable refresh period (default ~10Hz to match radar)
- Startup position: Center (90° or neutral position)
- Range requirement: $\pm 90^\circ$ from center (full 180° range not needed)
- Dual servos: Use tested configuration from servo_validation project
- Single MG996R: Needs range testing, but $\pm 90^\circ$ should be achievable

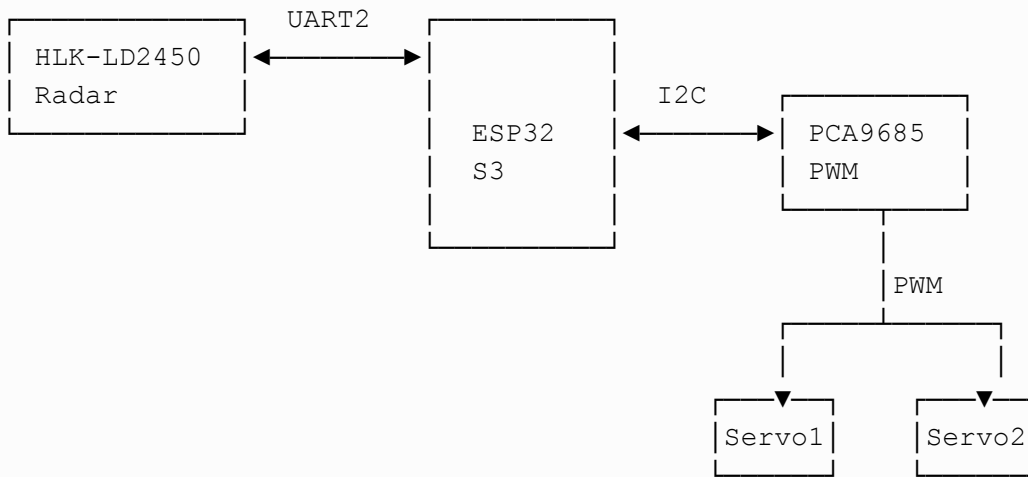
Development Philosophy:

- This is a demo - prioritize simplicity over completeness
- Minimal configuration of LD2450 sensor
- No complex calibration routines initially

System Overview

SimpleFaceMe integrates three main subsystems:

1. **HLK-LD2450 Radar Sensor** - Detects targets and provides X,Y coordinates
2. **PCA9685 PWM Driver** - Generates servo control signals via I2C
3. **Servo Motors** - Physical actuators that orient "heads" toward targets



Component Details

HLK-LD2450 Radar Sensor

- **Interface:** UART (TX/RX) at 256000 baud
- **Output:** Multi-target tracking data (X, Y coordinates in mm)
- **Range:** Configurable detection zone
- **Target Selection:** Uses primary/closest target for servo tracking

PCA9685 PWM Servo Driver

- **Interface:** I2C at 100kHz (default address 0x40)
- **PWM Frequency:** 50Hz for servo control
- **Channels:** 16 available (using 1 or 2 for this project)
- **Resolution:** 12-bit (0-4095)

Servos

- **Type:** MG996R (180° rotation, uncalibrated in this demo)
- **Control:** Standard PWM (500-2500µs pulse width)
- **Mounting:** Pan-only configuration (horizontal rotation)

Geometry Configuration

Coordinate System

- **Origin:** LD2450 sensor position
- **X-axis:** Left (-) to Right (+) when facing forward
- **Y-axis:** Backward (-) to Forward (+)
- **Z-axis:** Down (-) to Up (+)
- **Angles:** 0° = facing forward (+Y), positive rotation = counterclockwise from above

Single Servo Build

```
Sensor position: (0, 0, 0)
Servo axis:      (0, -50mm, 0)  // 5cm behind sensor
```

Configuration parameters:

```
- SENSOR_OFFSET_X = 0      (mm)
- SENSOR_OFFSET_Y = 0      (mm)
- SERVO_OFFSET_X   = 0      (mm)
- SERVO_OFFSET_Y   = -50    (mm)
```

Dual Servo Build

```
Sensor position: (0, 0, 0)
Servo 1 (left):  (-150mm, -100mm, 0)  // 15cm left, 10cm back
Servo 2 (right): (+150mm, -100mm, 0)  // 15cm right, 10cm back
```

Configuration parameters:

```
- SENSOR_OFFSET_X = 0      (mm)
- SENSOR_OFFSET_Y = 0      (mm)
- SERVO1_OFFSET_X = -150    (mm)
- SERVO1_OFFSET_Y = -100    (mm)
- SERVO2_OFFSET_X = 150     (mm)
- SERVO2_OFFSET_Y = -100    (mm)
```

Note: These are initial values and should be configurable via `build_flags` for easy adjustment without code changes.

Angle Calculation

For each servo, calculate the required pan angle to face a target at (target_x, target_y):

```
relative_x = target_x - servo_offset_x
relative_y = target_y - servo_offset_y
angle = atan2(relative_x, relative_y) * 180.0 / PI
```

The angle is then mapped to servo PWM values considering the servo's mechanical range and center position.

Data Flow

1. Radar Update (periodic, ~10Hz)

- Read UART data from LD2450
- Parse target data
- Select primary target (closest or strongest)

2. Angle Calculation

- Compute required servo angle(s) based on geometry
- Apply limits and safety checks

3. Servo Update

- Convert angle to PWM pulse width
- Send I2C commands to PCA9685
- Update servo positions

Safety & Limits

- **Mechanical stops:** Servos limited to safe range ($\pm 90^\circ$ from center)
- **Rate limiting:** Maximum servo speed to prevent damage
- **Timeout:** Return to center if no target detected for >5 seconds
- **I2C watchdog:** Reinitialize PCA9685 if communication fails

Build Configurations

Both builds share the same codebase with conditional compilation via build_flags:

- `SINGLE_SERVO_BUILD` - Enables single servo configuration
- `DUAL_SERVO_BUILD` - Enables dual servo configuration
- Geometry offsets defined per build
- Number of servos: `NUM_SERVOS` (1 or 2)